
Data analysis in Big Data environments: MongoDB

Exercises

Juan López Rubio juan.lopez-rubio@upc.edu

20-10-2023



Exercise 1: Metereological Database

You have been given a dataset containing meteorological data. The data is stored in a JSON file named `weather_data.json`. The structure of the data is as follows:

```
1 {  
2   "date": "2023-10-20",  
3   "temperature": 25.0,  
4   "humidity": 60.0,  
5   "wind_speed": 10.0  
6 }
```

Instructions:

1. Create a connection to a Mongo database named `meteo`.
2. Insert the JSON object provided as a example.
3. Generate a random dataset with 20 entries into a Python list. Assume that:
 - the values for temperature are between 20°C and 30°C.
 - the values for humidity are between 50% and 80%
 - and the values for wind speed are between 5 km/h and 15 km/h. The dates start the 2024-01-01 and increase by one day.
4. Insert the generated dataset into the database.
5. Query the database to retrieve the entries with a temperature higher than 30°C.
6. Query the database to retrieve the entries with a wind speed higher than 10 km/h.
7. Query the database to retrieve the entries with a humidity lower than 70%.
8. Query the database to retrieve the entries with a temperature between 20°C and 25°C.
9. Query the database to retrieve the entries with a wind speed higher than 10 km/h and a humidity lower than 60%.
10. Write this results into a JSON file named `weather_results.json`.
11. Update the database to add a new field named `rain` with a random value between 0 and 10.
12. Delete all the entries from may 2024.

Hints:

- Remember that the function `random.uniform(a, b)` generates a random float number between `a` and `b`.
- You can use the `datetime` module to generate the dates. For example:

```
1 import datetime  
2 date = datetime.datetime(2024, 1, 1)
```

```
3
4 next_date = date + datetime.timedelta(days=1)
5
6 # print next_date in format YYYY-MM-DD
7 print(next_date.strftime('%Y-%m-%d'))
```

Exercise 2: Cattle Census

You can use the provided JSON file `censo-ganado.json`. It's a simplified version of the dataset provided by the Instituto Cántabro de Estadística at:

<https://datos.gob.es/en/catalogo/a06004074-censo-de-ganado1>

Each line provides information about the number of cattle in a municipality in Cantabria. The structure of the data is as follows:

```
1 {
2   "Año": "2001",
3   "Especie": "Vacuno",
4   "Municipio": "39001 - Alfoz de Lloredo",
5   "Número de explotaciones": 170.0,
6   "Número de reses": 3829.0,
7   "Reses por explotación": 22.52
8 }
```

Instructions:

1. Create a connection to a Mongo database named `cattle`.
2. Read the JSON file and insert all the documents into the `census` collection.
Hint: You can rename the fields to make them more readable. For example, you can rename the field `Número de explotaciones` to `farms`.
3. Query the database to retrieve the entries with more than 1000 cattle.
Hint: You can convert the result to a DataFrame using the `pandas` library for simple visualization.
4. Query the database to retrieve the entries with more than 100 cattle and 10 farms.
5. Query the database to retrieve all the data for 'Ovino' cattle at the municipality of '39083 - Soba'.
6. Make a bar plot with this data to see how this information evolves over the years.
7. Update the database to split the field `Municipio` into two fields: `municipality_code` and `municipality_name`.
8. Write the final dataset into a JSON file named `cattle_results.json`.